Assignment 1: Report

Martina Carminati, Sanna Plomp, Keval Mehta

November 6, 2026

1 Learning the Basics of PyTorch

1.1 MLP

Started with a baseline model with hidden_size = 256, dropout = 0.2, optimiser = SGD, and learning_rate = 0.01. Used a batch of 64 and 10 epochs. Here, we varied the activation layers. Tried out ReLU, LeakyReLU, and GELU. Using a 90/10 split for training and validation, LeakyReLU resulted in the highest validation accuracy $0.\bar{9}018$.

Will continue varying the optimisers, using LeakyReLU as the activation function. Tried out SGD, Adam, and AdamW. Again using a 90/10 split for training and validation, AdamW resulted in the highest validation accuracy $0.\bar{8}650$.

Tried out learning rate [0.001, 0.01, 0.1]. Learning rate $0.\bar{0}1$ resulted in the highest validation accuracy of 0.9025.

Tried out hidden sizes of 128, 256, and 512. Hidden size of 128 was the best, with a validation accuracy of 0.8722.

Tried out dropout values of 0.0, 0.2, and 0.5. Dropout $0.\bar{0}$ resulted in the highest validation accuracy of 0.8788.

1.2 Convolutional Neural Networks

We started by setting seeds for reproducibility, and then importing the MNIST dataset from the natively provided `torchvision.datasets`. After importing the train and test separately, we split the training dataset into two subsets: what we will use for training (90%), and validation (10%). Setting batch size to 64, we created data loaders that will then load the data into the models.

We set up a basic convolutional neural network, similar to the one depicted in Figure 1, with two convolutional layers and max pooling between, with a customisable activation function.

Defining functions for training, validating, and testing, we then moved to the systematic testing of parameters.

We tested different activation functions:

ReLU

$$f(x) = \begin{cases} x & x \geqslant 0 \\ 0 & \text{otherwise} \end{cases}.$$

Leaky ReLU

$$f(x) = \begin{cases} x & x \geqslant 0 \\ mx & \text{otherwise} \end{cases}.$$

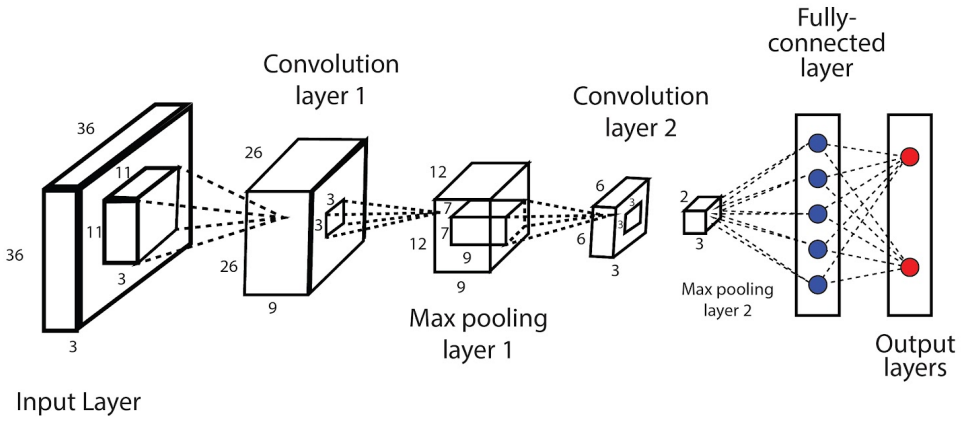


Figure 1: Sample Convolutional Neural Network

GeLU

$$f(x) = x \cdot \Phi(x).$$

We performed this testing with 10 epochs for training, which was validated through the separated dataset. The test performance of that model was then calculated. The performance of the different activation functions is given in Table 1. As we can see in the table, GELU performed best, so we will use it to fit the rest.

Activation	**Optimiser**	**Validation Accuracy**
ReLU	SGD	81.25%
Leaky ReLU	SGD	81.12%
GELU	SGD	81.75%

Table 1: Performance on validation dataset by different activation functions

From Table 2, we can see Adam does best, so we will use that in our further analysis. Now, we test different parameter combinations of the optimiser we have chosen, and the results are present in 3. We can see that the best performing model, as per our combined analysis, is with the GELU activation function, Adam optimiser, learning rate$= 10^{-3}$, weight decay$= 10^{-5}$, and betas$= (0.9, 0.999)$. We see a consistently strong performance with these values of learning rate and betas, even when weight decay is varied, even though that does decrease it marginally.

Further, after this, we ran the models on the CIFAR-10 dataset, adjusting them only for number of channels. Performance was not as strong, as demonstrated in Table 4. After running the three best models from

Activation	**Optimiser**	**Validation Accuracy**
GELU	SGD	81.75%
GELU	Adam	90.13%
GELU	AdamW	90.00%

Table 2: Performance on validation dataset by different optimisers

Activation	Optimiser	Validation Accuracy	Learning Rate	Weight Decay	Betas
GELU	Adam	0.9035	10^{-3}	10^{-5}	$(0.9, 0.999)$
GELU	Adam	0.9002	10^{-3}	10^{-5}	$(0.95, 0.995)$
GELU	Adam	0.8960	10^{-3}	10^{-5}	$(0.99, 0.99)$
GELU	Adam	0.9025	10^{-3}	10^{-4}	$(0.9, 0.999)$
GELU	Adam	0.8948	10^{-3}	10^{-4}	$(0.95, 0.995)$
GELU	Adam	0.8888	10^{-3}	10^{-4}	$(0.99, 0.99)$
GELU	Adam	0.8837	10^{-2}	10^{-5}	$(0.9, 0.999)$
GELU	Adam	0.8787	10^{-2}	10^{-5}	$(0.95, 0.995)$
GELU	Adam	0.8878	10^{-2}	10^{-5}	$(0.99, 0.99)$
GELU	Adam	0.8800	10^{-2}	10^{-4}	$(0.9, 0.999)$
GELU	Adam	0.8623	10^{-2}	10^{-4}	$(0.95, 0.995)$
GELU	Adam	0.8803	10^{-2}	10^{-4}	$(0.99, 0.99)$

Table 3: Performance on validation dataset by different parameters

Table 3, we tested different optimisers as well, to see more examples, as shown in Table 5. None of these performed particularly well.

Learning Rate	Weight Decay	Betas	Test Accuracy
10^{-3}	10^{-5}	$(0.9, 0.999)$	0.6216
10^{-3}	10^{-4}	$(0.9, 0.999)$	0.6172
10^{-3}	10^{-5}	$(0.95, 0.995)$	0.6407

Table 4: CIFAR-10 performance on the best three parameter combinations of `optim.Adam`

This could be due to the fact that in essence, MNIST is a much simpler task, where the digits are centred, and single-channel, and there is possibly a high level of separation between classes. The images of CIFAR-10 are not only RGB, but more complicated, and not as well separated. SGD performs much worse than Adam, which could be due to a weakly-trained learning rate on CIFAR, as opposed to Adam, which generalises better and isn't as dependent on the learning rate, as it uses the moments to adapt. While this difference is large, there is not much between Adam and AdamW, which may be due to both not overfitting, hence the decay is not a huge advantage.

Within the Adam optimiser, lowering the decay enables it to adapt faster, which may make it perform

Name	Test Accuracy
Adam	0.6288
AdamW	0.6289
SGD	0.4495

Table 5: CIFAR-10 performance on different optimisers

better on a complex dataset. I was not able to vary the architecture as much as I would've liked to, but adding more convolutional kernels may help the models to generalise better to the more complicated CIFAR-10 dataset.